Classic ML: K-Nearest Neighbours and K-Means Clustering

Module 4 of 4 — Python and Machine Learning with PyTorch

Sayan CHAKI

Chercheur
LIRIS, Ecole Centrale Lyon, Univ. Lyon 2, INSA

September 9, 2026

Table of Contents

Supervised versus unsupervised

KNN	**K-Means**
Supervised: needs labels y.	Unsupervised: only features X.
Task: classification (or regression).	Task: clustering.
k = number of neighbours consulted.	k = number of clusters produced.
No training phase at all.	Iterative training until convergence.
Prediction is expensive.	Prediction is cheap after fitting.

The k is not the same k

Same letter, completely different meaning. This confuses students every year.

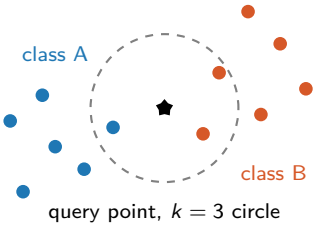
Both algorithms rest on one shared primitive: **a distance between points**.

Where we are

The idea in one sentence

KNN

To classify a new point, look at the k closest labelled points and take a majority vote.



Inside the circle: 1 blue, 2 orange, so the star is predicted as **class B**.

Two defining properties

Non-parametric

- The model has no weights to learn.
- The model **is** the training set.
- Complexity grows with the data, not with a fixed parameter count.

Lazy (instance-based)

- Training cost: $O(1)$, just store the data.
- All the work happens at prediction time.
- The opposite of a neural network, which is expensive to train and cheap to query.

Cost of predicting one point

With n training samples in dimension d: $O(nd)$ for the distances, plus $O(n \log k)$ for the selection. Predicting m points costs $O(mnd)$.

Distances

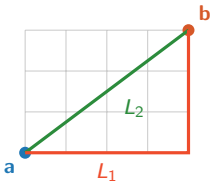
For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$:

$$\text{Euclidean } (L_2): \quad \sqrt{\sum_{j=1}^{d} (a_j - b_j)^2}$$

$$\text{Manhattan } (L_1): \quad \sum_{j=1}^{d} |a_j - b_j|$$

$$\text{Minkowski } (L_p): \quad \left(\sum_{j=1}^{d} |a_j - b_j|^p \right)^{1/p}$$

$$\text{Cosine:} \quad 1 - \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$



Scaling is not optional

A feature measured in millimetres dominates one measured in metres. Always standardise features before a distance-based method.

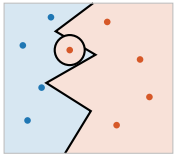
The KNN algorithm, step by step

1. **Fit:** store $X_{\text{train}} \in \mathbb{R}^{n \times d}$ and $y_{\text{train}} \in \mathbb{R}^{n}$. That is the entire training.
2. **Predict**, for each query $\mathbf{q}$:
   a. Compute the distance from $\mathbf{q}$ to all n training points.
   b. Sort and keep the k smallest.
   c. Read the labels of those k neighbours.
   d. Return the majority label (ties broken by the nearest neighbour).

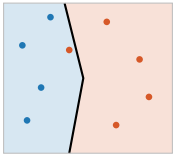
Vectorised version

Steps 2a and 2b are done for **all** queries at once with a single distance matrix of shape (m, n). No Python loop over samples.

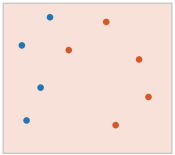
The effect of k on the decision boundary



$k = 1$: jagged, **overfits**, zero training error.



$k = 5$: smoother, usually the **sweet spot**.



$k = n$: everything is the majority class, **underfits**.

Choosing k

Small k means low bias and high variance. Large k means the opposite. Choose k by cross-validation, and prefer an odd k for two classes to avoid ties.

Strengths, weaknesses, and the curse of dimensionality

Strengths

- No training, trivially simple.
- Naturally multi-class.
- Can model arbitrarily complex boundaries.
- A strong baseline worth reporting.

Weaknesses

- Prediction is slow and memory-hungry.
- Sensitive to feature scale and to noise.
- Degrades badly in high dimension.

Curse of dimensionality

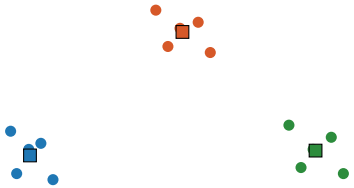
As d grows, the ratio between the farthest and the nearest distance tends to 1. Every point becomes roughly equidistant from every other, so "nearest" stops carrying information. Reduce dimension first, for example with PCA, or use a learned embedding.

Where we are

The idea in one sentence

K-Means

Split the data into k groups so that every point is as close as possible to the centre of its own group.



squares = centroids μ_1, μ_2, μ_3

The objective function

Given $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, find centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ and an assignment $c(i) \in \{1, \ldots, k\}$ minimising the **within-cluster sum of squares**:

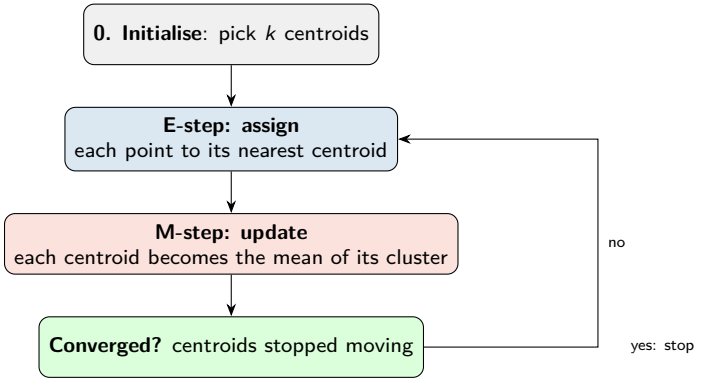
$$J = \sum_{i=1}^{n} \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|_2^2$$

- Also called **inertia**. It never increases across iterations.
- Finding the global optimum is NP-hard, so we use a greedy iterative scheme.
- That scheme is **Lloyd's algorithm**, and it is a form of expectation-maximisation.

Consequence

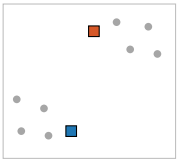
Different initialisations give different results. Run the algorithm several times and keep the run with the lowest J.

Lloyd's algorithm as expectation-maximisation

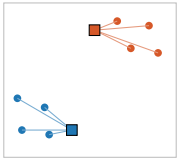


Each step can only decrease J, so the algorithm always terminates. It converges to a **local** optimum, not necessarily the global one.

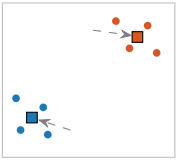
One iteration, drawn



Start: random centroids.



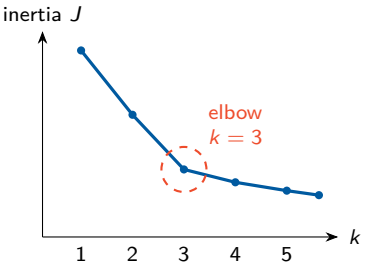
E-step: assign to nearest.



M-step: move to cluster means.

Repeat E and M until the centroids stop moving. Here, two iterations suffice.

Choosing k: the elbow method



- Run K-Means for $k = 1, 2, 3, \ldots$
- Plot the inertia J against k.
- J always decreases, and reaches 0 when $k = n$. So you cannot simply minimise it.
- Pick the **elbow**: the point after which adding a cluster buys almost nothing.

Alternatives: silhouette score, gap statistic, or domain knowledge, which is often the most reliable of the three.

Initialisation and limitations

k-means++ initialisation

Pick the first centroid uniformly at random. Then pick each subsequent centroid with probability proportional to $D(\mathbf{x})^2$, the squared distance to the nearest already-chosen centroid. This spreads the seeds out and converges much more reliably than pure random initialisation.

Where K-Means fails:

- Clusters that are not roughly spherical or that have very different sizes.
- Very different densities: the algorithm implicitly assumes equal variance.
- Outliers pull centroids away, because the mean is not robust.
- k has to be chosen in advance.

If these assumptions break, look at DBSCAN, Gaussian mixture models, or spectral clustering.

Where we are

The shared primitive: `torch.cdist`

```
A = torch.randn(5, 3)      # 5 query points in 3 dimensions
B = torch.randn(8, 3)      # 8 reference points

D = torch.cdist(A, B, p=2)     # Euclidean
print(D.shape)                 # torch.Size([5, 8])
# D[i, j] = distance between A[i] and B[j]

D1 = torch.cdist(A, B, p=1)    # Manhattan

# Equivalent manual version, useful to understand broadcasting:
diff = A.unsqueeze(1) - B.unsqueeze(0)    # (5,1,3) - (1,8,3) -> (5,8,3)
D_manual = diff.pow(2).sum(dim=2).sqrt() # (5,8)
```

Read the shapes

`unsqueeze(1)` and `unsqueeze(0)` set up the broadcast so that every query meets every reference. This one trick powers both algorithms.

KNN: pseudo-code and PyTorch side by side

Pseudo-code

1. $D \leftarrow$ distances between all queries and all training points
2. $I \leftarrow$ indices of the k smallest entries in each row of D
3. $L \leftarrow y_{\text{train}}[I]$
4. return the most frequent label in each row of L

```
class TorchKNN:
    def __init__(self, k=5, p=2):
        self.k, self.p = k, p

    def fit(self, X, y):
        self.X, self.y = X, y          # "training" is just storing
        return self

    def predict(self, Q):
        D = torch.cdist(Q, self.X, p=self.p)          # (m, n)
        idx = D.topk(self.k, dim=1, largest=False).indices   # (m, k)
        labels = self.y[idx]                          # (m, k)
        n_cls = int(self.y.max()) + 1
        votes = torch.zeros(Q.shape[0], n_cls, device=Q.device)
        votes.scatter_add_(1, labels, torch.ones_like(labels, dtype=votes.dtype))
        return votes.argmax(dim=1)                    # (m,)
```

Drawing the KNN decision boundary

```python
# 1. Build a dense grid covering the feature space
x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
xx, yy = torch.meshgrid(
    torch.linspace(x_min, x_max, 300),
    torch.linspace(y_min, y_max, 300),
    indexing="xy",
)
grid = torch.stack([xx.reshape(-1), yy.reshape(-1)], dim=1)   # (90000, 2)

# 2. Classify every grid point
Z = knn.predict(grid).reshape(xx.shape)

# 3. Paint the regions, then overlay the training data
plt.contourf(xx, yy, Z, alpha=0.3, cmap="coolwarm")
plt.scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm", edgecolors="k", s=25)
plt.title(f"KNN decision boundary (k={knn.k})")
```

Memory

A 300×300 grid against n training points builds a distance matrix with $90000 \times n$ entries. Process the grid in chunks if it overflows.

K-Means: pseudo-code and PyTorch

Pseudo-code

1. initialise $\mu_1 \dots \mu_k$
2. repeat: assign each point to the nearest centroid (E), then recompute each centroid as the mean of its members (M)
3. stop when centroids move less than `tol`

```
def kmeans(X, k=3, n_iter=100, tol=1e-4, seed=0):
    g = torch.Generator().manual_seed(seed)
    idx = torch.randperm(X.shape[0], generator=g)[:k]
    centroids = X[idx].clone()                          # (k, d)

    for it in range(n_iter):
        D = torch.cdist(X, centroids)                   # (n, k)   E-step
        assign = D.argmin(dim=1)                        # (n,)

        new_c = torch.stack([                           # M-step
            X[assign == j].mean(dim=0) if (assign == j).any() else centroids[j]
            for j in range(k)
        ])
        shift = (new_c - centroids).norm()
        centroids = new_c
        if shift < tol:
            break

    inertia = (X - centroids[assign]).pow(2).sum()
    return centroids, assign, inertia.item()
```

A fully vectorised M-step

The list comprehension above is clear but loops over k. Here is the version with no Python loop, which matters when k is large.

```
def m_step(X, assign, k):
    n, d = X.shape
    sums = torch.zeros(k, d, device=X.device)
    counts = torch.zeros(k, device=X.device)

    # scatter each point into the row of its cluster
    sums.index_add_(0, assign, X)
    counts.index_add_(0, assign, torch.ones(n, device=X.device))

    counts = counts.clamp(min=1)          # guard against empty clusters
    return sums / counts.unsqueeze(1)     # (k, d) broadcasting
```

Empty clusters

If no point is assigned to a centroid, its mean is undefined. Common fixes: keep the old centroid, or re-seed it at the point farthest from its centroid.

Complexity and a GPU note

	Time	Memory for the distance matrix
KNN prediction	$O(mnd)$	$O(mn)$
K-Means, one iteration	$O(nkd)$	$O(nk)$

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
X = X.to(device)          # a single .to() and everything runs on the GPU
centroids, assign, J = kmeans(X, k=3)
assign = assign.cpu()     # back to CPU for matplotlib
```

The payoff of writing it in PyTorch

Because everything is expressed as tensor operations, the same code runs on the GPU with no modification. A NumPy implementation would not.

Where we are

Colab Notebook 3: DataLoaders, EDA, KNN & K-Means

Part D — KNN

- Generate data with `sklearn.datasets.make_classification`.
- Implement `TorchKNN` from scratch with `torch.cdist` and `topk`.
- Compare L_1 and L_2, and sweep k to find the best value.
- Plot the decision boundary for several k and see overfitting appear.

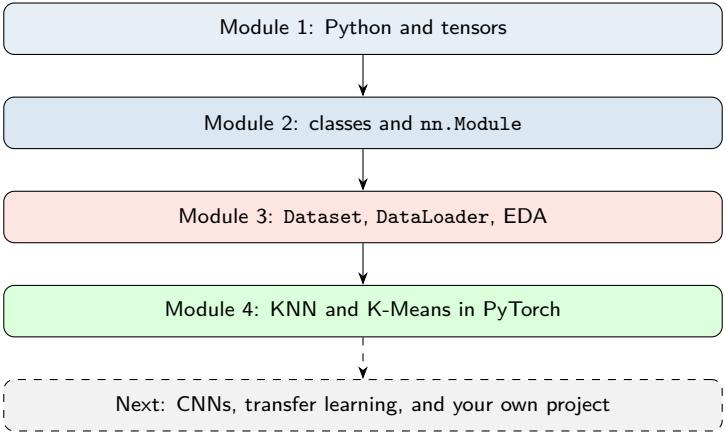
Part E — K-Means

- Generate blobs, implement Lloyd's algorithm in PyTorch.
- Track inertia across iterations and confirm it never increases.
- Plot the elbow curve and pick k.
- Visualise the clusters and the final centroids.
- **Challenge:** implement k-means++ initialisation and compare inertia.

Key takeaways

1. KNN is supervised and lazy: no training, expensive prediction, and the model is the data itself.
2. k controls the bias-variance trade-off; the decision boundary goes from jagged to flat as k grows.
3. K-Means is unsupervised and minimises within-cluster sum of squares by alternating an E-step and an M-step.
4. K-Means finds a local optimum, so initialisation matters; use k-means++ and several restarts.
5. Both reduce to one primitive, `torch.cdist`, plus `topk` or `argmin`.
6. Writing them in PyTorch means they run on the GPU for free.

End of the module



Thank you, and good luck with the practicals.